

CO4 -AT1-

NAME : GAYATHRI N P

REG NO : 192421362

- 1. Given a sequence of integers, determine the length of the longest subsequence in which the elements**

appear in strictly increasing order. The subsequence must preserve the original order of elements, but the

chosen elements need not be adjacent. The goal is to identify the maximum possible increasing

subsequence length. Use Dynamic Programming to solve the problem efficiently.

Sample Input:

n = 6

arr = 10 22 9 33 21 50

Expected Output:

LIS Length = 4

AIM

To develop a Longest Increasing Subsequence (LIS) Analyzer using Dynamic Programming that identifies the longest strictly increasing subsequence from a given sequence and displays its length.

OBJECTIVES

1. To implement the LIS problem using the Dynamic Programming technique.
2. To find the longest strictly increasing subsequence from a given sequence.
3. To maintain the original order of elements while selecting the subsequence.
4. To calculate the LIS value for each element using a DP table.
5. To avoid repeated calculations and improve the efficiency of the solution.
6. To identify the actual elements that form the longest increasing subsequence.
7. To display the DP values, previous elements, and selected elements clearly.
8. To provide a step-by-step explanation of how the LIS is calculated.
9. To develop an interactive application where users can enter their own sequence.
10. To display the final LIS and its maximum length in a clear and user-friendly interface.

ALGORITHM

1. Start the program.
2. Read the input sequence of integers.
3. Create a DP array and initialize every element to 1.
4. Create a parent array to store the previous element of the subsequence.
5. For every element $arr[i]$, compare it with all previous elements $arr[j]$.
6. If $arr[i] > arr[j]$, check whether extending the subsequence gives a larger value.
7. Update the DP value using:
 $DP[i] = \max(DP[i], DP[j] + 1)$
8. Store the index of the previous element in the parent array.
9. Find the maximum value in the DP array.
10. Trace the parent array backwards to reconstruct the actual LIS.
11. Display the DP table and selected LIS elements.
12. Display the Longest Increasing Subsequence and its length.
13. Stop the program.

Program :

```
import tkinter as tk

from tkinter import ttk

def analyze():
    a=list(map(int,entry.get().split()))
    n=len(a)
    dp=[1]*n
    parent=[-1]*n

    for i in range(n):
        for j in range(i):
            if a[i]>a[j] and dp[i]<dp[j]+1:
                dp[i]=dp[j]+1
```

```
parent[i]=j
```

```
k=dp.index(max(dp))
```

```
lis=[]
```

```
while k!=-1:
```

```
    lis.append(a[k])
```

```
    k=parent[k]
```

```
lis.reverse()
```

```
for x in table.get_children():
```

```
    table.delete(x)
```

```
for i in range(n):
```

```
    p="-" if parent[i]==-1 else a[parent[i]]
```

```
    s="LIS" if a[i] in lis else "Not Selected"
```

```
    table.insert("", "end", values=(i,a[i],dp[i],p,s))
```

```
result.config(text=" → ".join(map(str,lis)))
```

```
length.config(text="LIS LENGTH : "+str(len(lis)))
```

```
explanation.delete("1.0",tk.END)
```

```
explanation.insert(tk.END,"STEP-BY-STEP EXPLANATION\n\n")
```

```
for i in range(n):
```

```
    explanation.insert(tk.END,f"STEP {i+1}: Checking {a[i]}\n")
```

```
    if i==0:
```

```

        explanation.insert(tk.END,"First element  $\rightarrow$  DP = 1\n\n")
    else:
        for j in range(i):
            if a[i]>a[j]:
                explanation.insert(tk.END,f"{a[i]} > {a[j]}  $\checkmark$ \n")
            else:
                explanation.insert(tk.END,f"{a[i]} <= {a[j]}  $\times$ \n")

        explanation.insert(tk.END,f"DP[{i}] = {dp[i]}\n\n")

    explanation.insert(tk.END,"FINAL RESULT\n")
    explanation.insert(tk.END,f"LIS = {'  $\rightarrow$  '.join(map(str,lis))}\n")
    explanation.insert(tk.END,f"Length = {len(lis)}")

def reset():
    entry.delete(0,tk.END)
    for x in table.get_children():
        table.delete(x)
    result.config(text="---")
    length.config(text="LIS LENGTH : ---")
    explanation.delete("1.0",tk.END)

root=tk.Tk()
root.title("LIS Analyzer")
root.geometry("950x750")
root.configure(bg="#0F172A")

```

```
tk.Label(root,text="LONGEST INCREASING SUBSEQUENCE  
ANALYZER",font=("Arial",20,"bold"),bg="#172554",fg="white",pady=15).pack(fill  
="x")
```

```
frame=tk.Frame(root,bg="#1E293B",pady=15)  
frame.pack(fill="x",padx=20,pady=15)
```

```
tk.Label(frame,text="Enter  
Sequence:",bg="#1E293B",fg="white").pack(side="left",padx=10)
```

```
entry=tk.Entry(frame,width=50,bg="#334155",fg="white",insertbackground="white")  
entry.pack(side="left",padx=10,ipady=6)
```

```
tk.Button(frame,text="ANALYZE",command=analyze,bg="#60A5FA",fg="black").p  
ack(side="left",padx=5)
```

```
tk.Button(frame,text="RESET",command=reset,bg="#475569",fg="white").pack(side  
="left")
```

```
table=ttk.Treeview(root,columns=("Index","Element","DP","Previous","Status"),sho  
w="headings",height=5)
```

```
for c in ("Index","Element","DP","Previous","Status"):
```

```
    table.heading(c,text=c)
```

```
    table.column(c,anchor="center",width=170)
```

```
table.pack(fill="x",padx=20,pady=10)
```

```
result_frame=tk.Frame(root,bg="#102A43",pady=12)
```

```
result_frame.pack(fill="x",padx=20,pady=5)
```

```
tk.Label(result_frame,text="LONGEST INCREASING  
SUBSEQUENCE",bg="#102A43",fg="white").pack()
```

```
result=tk.Label(result_frame,text="---  
",bg="#102A43",fg="#4ADE80",font=("Arial",18,"bold"))  
result.pack()
```

```
length=tk.Label(result_frame,text="LIS LENGTH : ---  
",bg="#102A43",fg="#FACC15",font=("Arial",13,"bold"))  
length.pack()
```

```
exp_frame=tk.LabelFrame(root,text="STEP-BY-STEP  
EXPLANATION",bg="#1E293B",fg="#C084FC",font=("Arial",12,"bold"))  
exp_frame.pack(fill="both",expand=True,padx=20,pady=10)
```

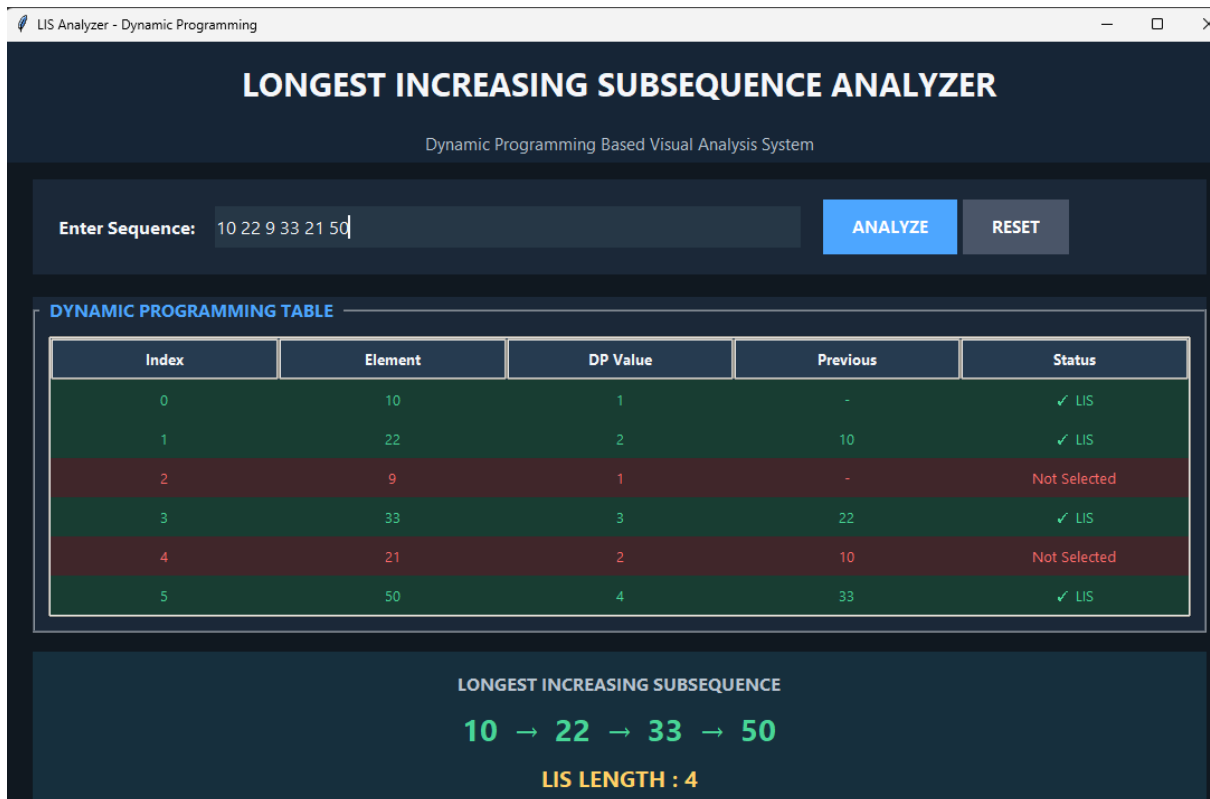
```
scroll=tk.Scrollbar(exp_frame)  
scroll.pack(side="right",fill="y")
```

```
explanation=tk.Text(exp_frame,bg="#111827",fg="white",font=("Arial",10),wrap="w  
ord",yscrollcommand=scroll.set)  
explanation.pack(fill="both",expand=True,padx=10,pady=10)
```

```
scroll.config(command=explanation.yview)
```

```
root.mainloop()
```

output :



RESULT

The Longest Increasing Subsequence (LIS) Analyzer was successfully implemented using the Dynamic Programming technique. For the given input sequence 10, 22, 9, 33, 21, 50, the program compares the elements in their original order and identifies the longest strictly increasing subsequence as 10 → 22 → 33 → 50. Therefore, the maximum length of the Longest Increasing Subsequence is 4. The program also displays the DP table, selected elements, and step-by-step analysis, making the calculation easy to understand.